

Name: Niraj Patil

Roll No: 381003

Batch: A1

Assignment No: 1

Problem Statement:

Implement a basic probabilistic generative model (e.g., Gaussian Mixture Models). Build a simple text generator using Markov Chains.

Objective:

- To understand the concept of probabilistic generative models and how they learn from training data.
- To study Markov Chains and the Markov property for modeling word transition probabilities in text.
- To implement a first-order Markov Chain text generator that trains on a corpus and generates new text sequences.
- To build a reusable MarkovTextGenerator class with train() and generate() methods.
- To test text generation by providing different starting words and analyzing the coherence of generated outputs.

S/W Packages and H/W apparatus used:

- Operating System: Windows/Linux/macOS
- Kernel: Python 3.x
- Tools: Jupyter Notebook / Anaconda
- Hardware: CPU with minimum 4GB RAM
- Libraries and packages used: Python random module, Python collections (defaultdict)

Theory:

Generative models are a class of machine learning models that learn the underlying probability distribution of training data and can generate new samples from that distribution. Unlike discriminative models that learn decision boundaries between classes, generative models learn how data is generated. Examples include Gaussian Mixture Models (GMMs), Hidden Markov Models (HMMs), Variational Autoencoders (VAEs), and Generative Adversarial Networks (GANs).

Markov Chains:

A Markov Chain is a stochastic process where the probability of transitioning to the next state depends only on the current state, not on the history of previous states. This is known as the Markov property or the memoryless property. In the context of text generation, each word represents a state, and the transition probability $P(\text{next word} \mid \text{current word})$ is estimated from the training corpus. A first-order Markov Chain considers only one preceding word as context, while higher-order chains consider N preceding words.

Training the Markov Model:

The training process scans through the tokenized corpus and, for every consecutive word pair (w_i , w_{i+1}), records w_{i+1} as a possible successor of w_i in a transition table implemented as a defaultdict of lists. Words that appear multiple times as successors of w_i are stored multiple times, so when a successor is sampled randomly from the list, more frequent transitions are naturally selected with higher probability — implicitly encoding Maximum Likelihood Estimation of transition probabilities.

Text Generation:

Given a start word, the generate() method iteratively samples the next word from the list of observed successors of the current word using random.choice(). If the current word has no recorded successors (a terminal state), a random word from the vocabulary is selected to continue generation. This process repeats until the desired sequence length is reached, and the tokens are joined into a readable text string.

Gaussian Mixture Models (GMM) — Background:

A Gaussian Mixture Model is a probabilistic generative model that represents the data distribution as a weighted sum of multiple Gaussian (normal) distributions, each with its own mean and covariance. GMMs are trained using the Expectation-Maximization (EM) algorithm, which alternates between assigning data points to mixture components (E-step) and updating component parameters (M-step). While Markov Chains model sequential data (text), GMMs model continuous feature distributions and are commonly used for clustering, density estimation, and anomaly detection.

Methodology:

- Import Python's random module and defaultdict from collections.
- Define the MarkovTextGenerator class with an __init__ method that initializes self.model as a defaultdict of lists.
- Implement the train(text) method: convert text to lowercase, split into words, and for each consecutive word pair (words[i], words[i+1]), append words[i+1] to self.model[words[i]].
- Implement the generate(start_word, length=30) method: initialize the generated list with the start word, then iteratively sample the next word using random.choice() from the model's successor list for the current word.
- If the current word has no successors, fall back to a random word from the model's vocabulary keys.
- Define a multi-sentence training corpus covering topics of deep learning, machine learning, data science, and AI.
- Instantiate MarkovTextGenerator, call train() with the corpus, and confirm successful training.
- Generate three text sequences starting from 'learning', 'data', and 'models' with length=12 and print the outputs.

Use Cases:

- Use Case 1 - Automated Content Drafting: Markov Chain text generators can produce rough drafts of domain-specific content (e.g., technical reports, product descriptions) by training on domain corpora, reducing the initial writing effort for content creators.
- Use Case 2 - Data Augmentation for NLP: Generated Markov Chain text sequences can augment small training datasets for text classifiers, providing additional synthetic training examples that mimic the statistical patterns of the original corpus.

- Use Case 3 - Chatbot Prototype Development: Simple Markov Chain generators serve as quick-to-build chatbot response prototypes for narrow domains, producing contextually plausible responses without requiring neural network training infrastructure.

Advantages:

- Extremely simple to implement with no external ML libraries — only Python's built-in random and collections modules are needed.
- Training is instant even on large corpora since the model only requires a single pass to build the transition table.
- The model is fully interpretable — the transition table explicitly shows which words follow which in the training data.
- Higher-order Markov Chains (bigram, trigram contexts) can be implemented with minimal code changes to improve coherence.
- The probabilistic sampling naturally produces varied outputs; each generation run produces a different text sequence.

Limitations:

- First-order Markov Chains have no memory beyond the immediately preceding word, often producing incoherent long sequences.
- The model cannot generalize to unseen word transitions; any word not in the training vocabulary triggers a random fallback.
- Generated text lacks global semantic coherence — sentences may be locally plausible but globally meaningless.
- The model does not handle sentence boundaries, punctuation, or grammar explicitly, resulting in run-on generated text.
- Quality is entirely limited by the size and diversity of the training corpus; small corpora lead to highly repetitive outputs.

Applications:

- Text autocomplete systems: Lightweight Markov models power next-word suggestions in resource-constrained environments.
- Creative writing assistance: Markov generators produce randomized story starters and plot prompts for writers seeking inspiration.
- Spam generation detection: Training spam detectors on Markov-generated synthetic spam improves robustness to novel spam patterns.
- Music and poetry generation: Applied to sequences of musical notes or poetic meters, Markov Chains generate stylistically consistent compositions.
- Game dialogue systems: NPCs in games use Markov Chain models to generate contextually varied dialogue from scripted training lines.

Conclusion:

This assignment implemented a first-order Markov Chain text generator using Python's defaultdict. A MarkovTextGenerator class was built with train() and generate() methods, trained on a six-sentence AI and data science corpus. The model successfully generated text sequences starting from 'learning', 'data', and 'models', producing outputs that reflect the statistical word transitions observed in the training data. While the generated text is locally coherent, the memoryless nature of first-order Markov Chains limits

global sentence coherence. This experiment establishes a clear understanding of probabilistic generative models and their transition from simple statistical approaches like Markov Chains to more powerful models such as Gaussian Mixture Models, Hidden Markov Models, and modern neural generative architectures.